Geavanceerde Computerarchitectuur — Session 1 notes

Slide 3 — GPUs can execute more threads simultaneously

What the slide shows

- One **process** with two thread pools: a small CPU pool and a much larger GPU pool.
- Left: the CPU threads are mapped onto a few large cores. Each core has its own control logic (yellow) and L1 cache (purple), the cores share L2/L3 caches, and everything sits on DRAM.
- Right: the GPU threads are mapped onto a large grid of small cores (green). There is only a thin strip of control and cache per row, then a shared L2 and DRAM.
- The figure comes from the CUDA C++ Programming Guide ("The GPU devotes more transistors to data processing").

Explanation

The design trade-off is how the transistor budget is spent.

	CPU	GPU
Optimized for	Latency (finish one thread fast)	Throughput (finish many threads per second)
Die area spent on	Control (branch prediction, out-of-order execution), large caches	ALUs
Threads	Tens	Tens of thousands resident
Hides memory latency by	Caches, speculation	Switching to another ready group of threads

- **Latency hiding.** When a group of GPU threads waits roughly 400–800 cycles for DRAM, the scheduler runs another group in the next cycle. This switch is free, because every resident thread keeps its own registers in the register file; nothing is saved or restored. This is why a GPU needs many threads and does not need big caches.
- **"Core" means different things.** A "CUDA core" is one FP32 ALU lane, not a core in the CPU sense. The real counterpart of a CPU core is the **SM** (Streaming Multiprocessor): one row of the green grid together with its control and cache strip.
- **Threads run in groups.** GPU threads execute in groups of 32 (**warps**) that share one instruction stream (SIMT). This is covered in a later session ("Warps" on slide 8).
- **Scale example (RTX 4090, compute capability 8.9).** It has 128 SMs, 16,384 FP32 lanes, and at most 1,536 resident threads per SM, so about 196k threads can be resident at once. A desktop CPU runs roughly 16–32 hardware threads.
- **"Same process".** The GPU is a coprocessor. The CPU program owns the process and offloads work to the GPU, and a discrete GPU has its own memory. This leads into slide 5.

Key points for the students

1. A CPU is a few strong cores and a GPU is many weak lanes. They answer different optimization goals: latency versus throughput.
2. A GPU only pays off when there are thousands of independent tasks.
3. Electronics angle: once Dennard scaling ended, power became the limit, so the way to get more performance is more parallel ALUs per watt, not higher clock frequency.

Likely questions

- **Why not run everything on the GPU?** Serial code, branch-heavy code and small problems run badly on a GPU, and copying data over PCIe has a cost. Amdahl's law limits the gain: speedup

= 1 / ((1−p) + p/N). With p = 0.9, the maximum is 10× no matter how many cores you add.
- **Is a GPU core slower than a CPU core?** Per thread, yes: lower clock, no out-of-order execution, in-order issue per warp. In aggregate throughput, the GPU is far faster.
- **Can CPU threads not also be switched?** They can, but an OS context switch costs microseconds. A warp switch on a GPU costs zero cycles.

Study instructions

1. Read the introduction of the CUDA C++ Programming Guide, the part titled “The Benefits of Using GPUs”, where this figure comes from.
2. On a lab machine, query the GPU with this program:

```
#include <cstdio>
int main() {
    cudaDeviceProp p;
    cudaGetDeviceProperties(&p, 0);
    printf("%s cc %d.%d SMs %d maxThr/SM %d maxThr/block %d\n",
           p.name, p.major, p.minor, p.multiProcessorCount,
           p.maxThreadsPerMultiProcessor, p.maxThreadsPerBlock);
}
```

   Compile it with `nvcc -o q q.cu && ./q`. Compute SMs × maxThr/SM and compare the result with `nproc` on the CPU. Keep these numbers; you will need them for slide 18 (maximum block size).
3. Be able to explain latency hiding without notes, using a concrete example: one warp waits on memory while the scheduler picks another ready warp.
4. Self-check: why does a GPU with little cache still reach high memory throughput? (Answer: many warps in flight keep the memory bus busy, and coalesced access, covered in a later session.)

Slide 4 — Example: parallel processing for ray tracing

What the slide shows

- A ray-traced render of many spheres with diffuse, metal (mirror) and glass materials. It is the classic “Ray Tracing in One Weekend” scene, here from the GPU ray tracer `mellinoe/veldrid-raytracer` (credited on the last slide).
- The point of the slide: this is a workload that matches slide 3 exactly.

Explanation

Why ray tracing parallelizes well:

- The color of every pixel is computed **independently**. A ray goes from the camera through the pixel, the code finds the nearest sphere it hits, and then the ray bounces: scattered for diffuse surfaces, reflected for metal, refracted for glass. This repeats until the ray escapes or a depth limit is reached.
- Pixels do not depend on each other, so the work maps to **one thread per pixel**. A 1920×1080 image is about 2M pixels, and with 100 samples per pixel that is about 200M independent rays.
- This kind of workload is called **embarrassingly parallel**: no communication and no synchronization between threads.

How it maps onto CUDA (this previews slides 12–18):

```cpp
__global__ void render(float3* img, int w, int h) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= w || y >= h) return;
    img[y * w + x] = trace(camera_ray(x, y));
}

dim3 threads(16, 16);
dim3 blocks((w + 15) / 16, (h + 15) / 16);
render<<<blocks, threads>>>(img, w, h);
```

- `trace` and `camera_ray` would be `__device__` functions (slide 5).
- The `if` guard is needed because the image size is rarely a multiple of the block size.

Where it gets less ideal. These are hooks for later sessions:

- **Divergence.** Neighbouring pixels can hit different materials or bounce a different number of times. Threads in the same warp then follow different paths and are serialized ("Warps" session).
- **Randomness.** Every thread needs its own random-number state (cuRAND).
- **Memory.** All threads read the same scene data, which suits constant or shared memory ("Memory" session).
- **Dedicated hardware.** Since Turing, NVIDIA GPUs also have RT cores for ray–triangle intersection, used through OptiX or Vulkan/DirectX RT. The example on this slide uses only general-purpose cores.

Key points for the students

1. Parallelism comes from **independent work items**. Before anything else, find the item: a pixel, a sample, a filter output, a packet.
2. Examples from electronics with the same structure: FIR or FFT over many channels, beamforming per angle, radio propagation ray tracing per ray, Monte Carlo simulation, image and video processing, block encryption (AES-CTR). Several of these appear as project topics on slide 8.
3. Counter-example: a running sum or an IIR filter. Each output depends on the previous one, so the naive version is not parallel. It needs a different algorithm, such as a scan or reduction.

Likely questions

- **Why one thread per pixel and not one per ray bounce?** The bounces of one ray are sequential, because each depends on the previous hit.
- **What if there are more pixels than threads?** Launch more blocks, since the grid can be huge, or use a grid-stride loop.

Study instructions

1. Read "Accelerated Ray Tracing in One Weekend in CUDA" (Roger Allen, NVIDIA Developer Blog). It ports Shirley's book to CUDA chapter by chapter. Run at least its first chapter, which outputs a gradient image with one thread per pixel.
2. Exercise: write the 2D kernel above with `trace` replaced by a gradient color. Compare it with a single-threaded CPU version timed with `std::chrono` at 4K resolution. Note that the first comparison is misleading if you include the memory transfer; that comes back on slide 5.
3. Be able to classify these as parallel or not, and why: element-wise vector addition, histogram, prefix sum, matrix multiply, and an IIR filter.

Slide 5 — Relationship between architecture and scope

(Three overlays: the base slide, then arrows to the GPU, then arrows for the call chain.)

What the slide shows

A single source file, `code.cpp`, has five parts. The slide marks which hardware each part runs on.

1. Includes and defines
2. CPU functions, kernel functions (`__global__`) and GPU functions (`__device__`, in green)
3. Memory allocation and migration
4. Kernel invocation (and timing)
5. Memory migration (and checking/dumping)

- **Overlay 1.** The whole file is compiled and executed **on the CPU**.
- **Overlay 2.** Kernel and device functions go to the GPU. Steps 3, 4 and 5 are the **interface to the GPU**: they run on the CPU but drive the GPU.
- **Overlay 3.** Step 4 (invocation) calls the kernel, and the kernel calls the device functions.

Explanation

Function qualifiers decide where code runs:

Qualifier	Runs on	Called from	Notes
none / `__host__`	CPU	CPU	Normal C++
`__global__`	GPU	CPU (or GPU via dynamic parallelism)	This is a **kernel**: returns `void`, launched with `<<<grid, block>>>`, asynchronous
`__device__`	GPU	GPU	Helper function called by kernels
`__host__ __device__`	both	both	Compiled twice

Compilation. `nvcc` splits the file. Host code goes to `g++`, and device code goes to PTX and then SASS (GPU machine code). Both end up in one executable (a "fat binary"). Two details to check:

- `nvcc` decides how to compile by file extension. A `.cpp` file is treated as plain host C++ and fails on `__global__` and `<<<>>>` unless it is compiled with `-x cu` or renamed `.cu`. Check what the lab's `tasks.json` does.
- To target the lab GPU explicitly, pass `-arch=sm_XY`, where XY is the compute capability (slide 6).

The five steps as a minimal program. This uses Unified Memory, which matches the word "migration" on the slide:

```
#include <cstdio>
#define N 1024

__device__ float sq(float x) { return x * x; }

__global__ void square(float* a, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) a[i] = sq(a[i]);
}

int main() {
    float* a;
```

```
    cudaMallocManaged(&a, N * sizeof(float));
    for (int i = 0; i < N; i++) a[i] = i;

    cudaEvent_t t0, t1;
    cudaEventCreate(&t0);
    cudaEventCreate(&t1);
    cudaEventRecord(t0);
    square<<<(N + 255) / 256, 256>>>(a, N);
    cudaEventRecord(t1);
    cudaEventSynchronize(t1);
    float ms;
    cudaEventElapsedTime(&ms, t0, t1);

    for (int i = 0; i < N; i++)
        if (a[i] != (float)i * i) { printf("mismatch at %d\n", i); break; }
    printf("%.3f ms\n", ms);
    cudaFree(a);
}
```

Explicit-copy variant of steps 3 and 5. This is the classic model, also worth knowing:

```
float *h = (float*)malloc(bytes), *d;
cudaMalloc(&d, bytes);
cudaMemcpy(d, h, bytes, cudaMemcpyHostToDevice);
kernel<<<g, b>>>(d, N);
cudaMemcpy(h, d, bytes, cudaMemcpyDeviceToHost);
cudaFree(d);
```

- **Unified Memory** (`cudaMallocManaged`) gives one pointer that is valid on both sides. The driver *migrates* pages on demand when they are touched, and `cudaMemPrefetchAsync` migrates them ahead of time.
- **Explicit copies** (`cudaMalloc` + `cudaMemcpy`) make every transfer visible and under your control.

Why step 4 says "and timing". A kernel launch is **asynchronous**: the CPU continues immediately. A CPU timer around the launch therefore measures only the launch overhead. Use CUDA events, as above, or synchronize before stopping the timer.

Why step 5 says "checking/dumping". GPU errors are silent unless you ask for them. Check `cudaGetLastError()` after a launch and the return value of `cudaDeviceSynchronize()`. Compare the result against a CPU reference, or dump it to a file.

```
#define CK(x) do { cudaError_t e = (x); if (e != cudaSuccess) { \
    printf("%s:%d %s\n", __FILE__, __LINE__, cudaGetErrorString(e)); exit(1); } } while (0)
```

Key points for the students

1. One file, two processors. **Qualifiers** decide where each function runs.
2. The host program is a **controller**. It allocates memory, moves data, launches the kernel, waits, and collects the result. Electronics analogy: a microcontroller driving a DMA peripheral, which means configure the buffers, start, wait for the done flag or interrupt, then read the buffer.
3. Kernel launches are asynchronous. Synchronize before using the results or reading a timer.

Likely questions

- **Can a kernel return a value?** No, it must return `void`. Results are written to memory through pointers.
- **Can a kernel call `printf`?** Yes. Output is buffered and flushed at synchronization points.

- **Can host code call a `__device__` function?** No, that is a compile error.

Study instructions

1. Type the minimal program, without copy-paste, and compile it with `nvcc -o sq sq.cu`.
2. Break it on purpose, one change at a time, and observe what happens:
   - Remove `cudaEventSynchronize`/`cudaDeviceSynchronize` before the check. Depending on the platform, the check reads results before the kernel has finished, or it segfaults.
   - Call `sq` from `main`. You get a compile error.
   - Launch with 2000 threads per block, then check `cudaGetLastError()`. This shows "invalid configuration" and links to slide 18.
   - Remove the `if (i < n)` guard and use N = 1000. The kernel writes out of bounds; run it with `compute-sanitizer ./sq` to catch this.
3. Time it twice, with `std::chrono` around the launch and with events, and explain the difference.
4. Rewrite the program with explicit `cudaMalloc`/`cudaMemcpy`, and time the copies separately. For small N, the copies dominate the run time.
5. Optional: run `cuobjdump -sass sq` to see the GPU machine code that `nvcc` embedded.

Slide 6 — CUDA, an API for programming Nvidia GPUs

What the slide shows

The "GPU Computing Applications" stack from the CUDA Programming Guide, read from top to bottom:

1. **Applications**
2. **Libraries and middleware:** cuDNN, TensorRT, cuFFT, cuBLAS, cuRAND, cuSPARSE, CULA, MAGMA, Thrust, NPP, VSIPL, SVM, OpenCurrent, PhysX, OptiX, iRay, MATLAB, Mathematica
3. **Programming languages:** C, C++, Fortran, Java/Python wrappers, DirectCompute, directives (OpenACC)
4. **CUDA-enabled NVIDIA GPUs:** a table with architecture generation as rows (Ampere 8.x, Turing 7.x, Volta 7.x, Pascal 6.x) and market segment as columns (Embedded/Jetson, Consumer GeForce, Professional Quadro, Data center Tesla)

Explanation

CUDA is more than a language extension. It is a whole platform:

- **Programming model:** kernels, a grid of blocks of threads, and a memory hierarchy.
- **Compiler:** `nvcc`.
- **Two APIs.** The **runtime API** (`cuda*` calls, `<<<>>>` syntax) is what this course uses. The **driver API** (`cu*` calls) is lower level and more explicit, and you rarely need it.
- **Libraries:** the second layer of the figure. The rule of thumb: if cuBLAS, cuFFT or Thrust already does it, use the library. Hand-written kernels are for what the libraries do not cover, or for learning, as in this course.
- **Tools:** `compute-sanitizer` (memory errors and race conditions), `cuda-gdb`, and Nsight Systems / Nsight Compute (profiling).

Compute capability (CC). CC is written `major.minor` and identifies the architecture generation. It determines the hardware limits and the features available, such as maximum threads per block, shared memory size, and support for dynamic parallelism. The tables at the end of the Programming Guide are indexed by CC. The question on slide 18 ("largest block size") is answered there: 1024 for all current CCs.

The figure is outdated. The deck is from 2023/24 and the figure is even older. The current situation:

- Generations after Ampere: Ada Lovelace (8.9, RTX 40), Hopper (9.0, H100/H200), Blackwell (10.x data center, 12.x RTX 50/workstation).
- **CUDA 13.0 removed offline compilation and library support for Maxwell, Pascal and Volta (CC < 7.5).** The Pascal and Volta rows of the figure, including the Jetson AGX Xavier, can no longer be targeted with CUDA 13. Use CUDA 12.x for those.
- The brand names "Tesla" and "Quadro" have been retired. Data-center GPUs are now named A100, H100 and so on, and professional cards are the RTX A-series / RTX Ada / RTX PRO lines.

Portability. CUDA runs only on NVIDIA GPUs. The alternatives:

- **OpenCL:** cross-vendor, more verbose.
- **HIP:** AMD's ROCm. Its API is nearly 1:1 with CUDA, and `hipify` converts code.
- **SYCL:** Intel and others.
- **Vulkan compute.**
- The concepts transfer directly: grid/block becomes NDRange/work-group in OpenCL, and warp becomes wavefront on AMD.

Key points for the students

1. CUDA = platform + programming model + runtime API + libraries + tools. In the course they write C++ against the runtime API.
2. Look up the lab GPU's compute capability first. It decides which limits apply and which `-arch` flag to use.
3. In practice, check for an existing library before writing a kernel.

Likely questions

- **Is CUDA a language?** It is C++ with extensions (qualifiers, `<<<>>>`, built-in variables) plus a runtime library.
- **Can CUDA be used from Python?** Yes: CuPy, Numba, the official `cuda-python` bindings, and PyTorch/JAX under the hood. That is the "wrappers" box in the figure.
- **Does a program compiled for one GPU run on another?** Usually. If PTX is embedded, the driver JIT-compiles it for newer GPUs. Native SASS only runs on the same major CC.

Study instructions

1. On a lab machine, run each of these and note the output:

   ```
   nvidia-smi
   nvidia-smi --query-gpu=name,compute_cap --format=csv
   nvcc --version
   nvcc --list-gpu-arch
   ```

   Check that the lab GPU's CC appears in the `--list-gpu-arch` output of the installed toolkit. If the lab still has Pascal or Volta cards, the toolkit must be 12.x.

2. In the Programming Guide, find the table of technical specifications per compute capability. Look up the lab GPU's values for: max threads per block, max resident threads per SM, max blocks per SM, and shared memory per block. This answers slide 18 and prepares the later sessions.

3. To see the library layer in action, sort 10M integers with Thrust (`thrust::device_vector` + `thrust::sort`, about 5 lines) and compare against `std::sort`.

4. Skim the Programming Model chapter of the Programming Guide (kernels, thread hierarchy, memory hierarchy). It covers slides 12–18 of this session.

Slide 7 — Environment and workflow

What the slide shows

1. Your code (C++) goes into the IDE (VS Code).
2. The IDE is driven by **instructions** (`tasks.json`) and calls the **compiler** (`nvcc`).
3. The result runs on the CPU, which uses the GPU.
4. Output from the CPU goes to the **terminal** (`printf`), a **dump** (file), or "Virus (…)".

Explanation

- **`tasks.json`** is VS Code's task configuration. It holds the exact `nvcc` command, so students don't type it, and it decides which flags are used (`-x cu`, `-arch`, `-g -G` for debugging, `-O3`). A minimal example:

```
{
  "version": "2.0.0",
  "tasks": [{
    "label": "build",
    "type": "shell",
    "command": "nvcc",
    "args": ["-x", "cu", "-arch=native", "-o", "${fileDirname}/${fileBasenameNoExtension}",
    ↪  "${file}"],
    "group": { "kind": "build", "isDefault": true }
  }]
}
```

- **The GPU has no I/O of its own.** The arrow goes from the GPU to the CPU, and every result reaches the outside world through the host. `printf` inside a kernel writes to a device buffer, which the host flushes at synchronization points. If the program exits without synchronizing, kernel `printf` output is lost.
- **A dump file** is the realistic way to verify large outputs: write the array to a file and compare or plot it offline (Python/numpy).
- **"Virus (…)"** is not explained on the slide. It is probably a joke for "other side effects". Ask the slide author how it is meant to be presented.

Key points for the students

1. Write code, build through the task, run, then verify the output via the terminal or a file.
2. The GPU never talks to the outside world directly.

Study instructions

1. Get the course's `tasks.json` from Toledo and explain every argument in it. Check how it handles `.cpp` versus `.cu` (see slide 5).
2. Add a `printf("%d\n", threadIdx.x);` to a kernel and run it with and without `cudaDeviceSynchronize()` at the end of `main`. Observe the output order: it is not sorted.
3. Write a 3-line dump (`fopen`/`fwrite`) and read it back with `numpy.fromfile`.

Slide 8 — Organization of the lab and docs

What the slide shows

The GCA lab has three branches:

- **Docs (Toledo):** programming guide, cheat sheet, `tasks.json`.
- **6 sessions:** Kernel, Race conditions, Warps, Memory, Async, Dynamic parallelism.
- **Project:** Encoding, Encryption, DSP, Graphics, and more.

Explanation: what each session covers

Session	Core concepts
Kernel	`__global__`, launch configuration, indexing, memory migration (this session)
Race conditions	Threads writing the same address; `atomicAdd`, `__syncthreads()`, reductions
Warps	Groups of 32 threads in lockstep (SIMT), branch divergence, warp shuffles
Memory	Global/shared/constant/registers, coalescing, shared-memory bank conflicts
Async	Streams, overlapping copies with compute, `cudaMemcpyAsync`, pinned host memory
Dynamic parallelism	Kernels launching kernels from device code

- **Dynamic parallelism changed with CUDA 12 (CDP2):** device-side `cudaDeviceSynchronize()` is no longer supported. If the old material for that session uses it, the material needs updating.
- **The project topics** are all embarrassingly or partially parallel workloads (slide 4): block ciphers, FIR/FFT, image and graphics kernels, and entropy or channel coding.

Study instructions

1. Download everything from Toledo and map each session to its chapter in the Programming Guide.
2. Read the session-2 material now: race conditions are the direct follow-up to the overwriting problem on slide 16.
3. Check the dynamic-parallelism material against the current toolkit version.

Slide 9 — Responsibilities and evaluation

What the slide shows

- **Lab sessions, 50%:** six single-page reports, one per session.
- **Project, 50%:** project report, project presentation, code release.

Explanation

A good single-page lab report contains:

1. The problem in one line.
2. The launch configuration and indexing formula, with a justification.
3. How correctness was verified (CPU reference, edge cases).

4. Timing, if relevant, with the method (CUDA events).
5. Answers to the open questions from the slides.

Study instructions

1. Write the report for session 1 yourself before grading any. It calibrates what is achievable on one page.
2. Ask the course owner for the grading criteria (rubric) for the reports and the project, if one exists.

Slide 10 — Recap

What the slide shows

Slides 5, 7, 8 and 9 side by side: code structure, workflow, lab organization and evaluation.

Study instructions

Without looking at the slides, be able to explain:

- the five parts of a CUDA program, and which processor each part runs on;
- the build and run path from the source file to the output;
- the six session topics, in order.

Slide 12 — Threads are organized into a grid

What the slide shows

- A pool of threads becomes a **grid** when the kernel is launched: `foo<<<2, 2>>>(arg1, arg2, ...)` creates two blocks (red) with two threads each (blue).

Explanation

- **Execution configuration:** `<<<gridDim, blockDim>>>`. There are two optional extra arguments, shared memory bytes and stream: `<<<g, b, smem, stream>>>`. Arguments can be `int` (1D) or `dim3` (2D/3D).
- **Hierarchy:** grid → blocks → threads. Total threads = gridDim × blockDim.
- **Why blocks exist:**
  - A block runs entirely on **one SM**. Its threads can cooperate through shared memory and `__syncthreads()`.
  - Blocks are **independent**. They can run in any order, on any SM, so the same binary scales from a GPU with 2 SMs to one with 140. Because of this, blocks cannot synchronize with each other inside a kernel.
- **Limits:**
  - A block has at most 1024 threads (x, y ≤ 1024; z ≤ 64; product ≤ 1024).
  - The grid can be up to $2^{31}-1$ blocks in x, and 65535 in y and z.
- **Warps:** the hardware splits each block into warps of 32. A block of 2 threads still occupies a whole warp, so 30 of its lanes are idle. Block sizes should therefore be multiples of 32.

Key points for the students

1. The launch configuration defines how many threads exist and how they are grouped.
2. Blocks are the unit of scheduling, and threads within a block are the unit of cooperation.

Likely questions

- **Why not one huge block?** One block runs on only one SM, the rest of the GPU sits idle, and a block is capped at 1024 threads.
- **Why not blocks of 1 thread?** Each block occupies a full warp of 32 lanes, so 31/32 of the hardware is wasted, and there is a limit on resident blocks per SM.

Study instructions

1. Launch `foo<<<2, 2>>>` with a kernel that prints `blockIdx.x` and `threadIdx.x`, then try `<<<4, 8>>>`. Note that the print order between blocks changes from run to run.
2. Try `dim3 b(16, 16); dim3 g(2, 3);` and print all four built-ins in x and y.

Slide 13 — Calling a kernel function: example

What the slide shows

- **Goal:** process each of 12 input elements with exactly one thread. The slide notes this is not always the case.
- **Open questions:**
  - OQ1: how do you choose the block and grid size?
  - OQ2: how does a thread know which element to process?

Explanation

- **One thread per element** is the default mapping. Alternatives come later:
  - one thread per several elements (grid-stride loop, slide 17);
  - several threads per element (reductions, session 2).
- OQ1 and OQ2 are the two decisions behind every kernel launch. Slides 14–18 answer them.

Study instructions

1. Before reading on, write down your own answers to OQ1 and OQ2. Students will be asked the same.

Slide 14 — OQ1: You decide how to divide the grid

What the slide shows

`foo<<<1, 12>>>` (1 block × 12 threads) and `foo<<<3, 4>>>` (3 blocks × 4 threads) both give 12 threads. Each thread is numbered t0–t11, or t0–t3 per block.

Explanation

- The programmer chooses the split, and both are functionally possible. The choice affects performance and limits, though:
  - `<<<1, 12>>>` uses one SM and cannot grow beyond 1024 elements.
  - `<<<3, 4>>>` spreads the work over 3 SMs, but every block of 4 threads occupies a full 32-lane warp.
- **Rule of thumb:**

```
int threads = 256;
int blocks = (n + threads - 1) / threads;
kernel<<<blocks, threads>>>(...);
```

  - Use a block size that is a multiple of 32, typically 128–512.
  - The grid size follows from N by rounding up (ceil division).
  - Rounding up creates surplus threads, so the kernel needs a bounds check.
- `cudaOccupancyMaxPotentialBlockSize` can suggest a block size automatically.

Likely questions

- **Which split is fastest?** It depends on registers and shared memory per thread (occupancy). Measure it; this is covered in the Memory and Warps sessions.

Study instructions

1. For N = 12, 1000 and 10^7, compute blocks and threads with the rule above and count the surplus threads.
2. Time a trivial kernel on 10^7 elements with block sizes of 32, 64, 128, 256, 512 and 1024 using CUDA events. Plot the results: the curve flattens.

Slides 15–16 — OQ2: Importance of correct indexing

What the slides show

- **Slide 15:** `<<<1, 12>>>`. Thread t_i handles in_i: good, one thread per element.
- **Slide 16:** `<<<3, 4>>>` with the index taken as `threadIdx.x` only. Thread t0 of every block maps to in0, t1 to in1, and so on. Elements in0–in3 are each handled by 3 threads, and in4–in11 are never touched.

Explanation

- `threadIdx.x` is the position **within the block**, so it repeats 0–3 in every block.
- **Overwriting:**
  - For pure writes such as `out[i] = f(in[i])`, the result is the same value written three times. It is wasteful, but the output for in0–in3 is still correct.
  - For read-modify-write operations (`a[i] += 1`) or in-place operations (flip), the result is wrong and **non-deterministic**. This is a race condition and leads into session 2.
- **Skipping:** two-thirds of the output is never computed and holds whatever was in memory before.
- **The bug is invisible with one block.** The code works with `<<<1, N>>>`, so students often only find it when N grows past 1024.

Study instructions

1. Write the buggy kernel `a[threadIdx.x] += 1` on an array of zeros and launch it with `<<<3, 4>>>`. The output is 3,3,3,3,0,0,0,0,0,0,0,0 in practice, but that is not guaranteed.
2. Explain to yourself why `+=` from different blocks can lose updates. The answer: a non-atomic read-modify-write.

Slide 17 — OQ2: Correct indexing

What the slide shows

- The thread's position within its block is only a relative position.
- Adding an offset of blockIdx × block size gives the absolute position: blocks 0, 1 and 2 map to elements 0–3, 4–7 and 8–11.

Explanation

The global index formula:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) out[i] = f(in[i]);
```

- The **bounds check** is mandatory because the grid size is rounded up (slide 14). Without it, the surplus threads write out of bounds. The program might not crash, but it corrupts memory.
- **2D version** (images, matrices):

  ```
  int x = blockIdx.x * blockDim.x + threadIdx.x;
  int y = blockIdx.y * blockDim.y + threadIdx.y;
  if (x < w && y < h) img[y * w + x] = ...;
  ```

- **Grid-stride loop.** Each thread handles several elements, so the grid size no longer has to match N:

  ```
  for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n; i += blockDim.x * gridDim.x)
      out[i] = f(in[i]);
  ```

Key points for the students

1. Global index = block offset + local index.
2. Always add the bounds check.

Study instructions

1. Fix the buggy kernel from slides 15–16, then test it with N = 13, which is not a multiple of 4.
2. Run it without the bounds check under `compute-sanitizer` and read the error report.
3. Implement the grid-stride version and launch it with a fixed grid (e.g. `<<<32, 256>>>`) for N = 10^7.

Slide 18 — Useful built-in functions

What the slide shows

- `threadIdx.x`: the thread's index within its block.

- `blockIdx.x`: the block's index within the grid.
- `blockDim.x`: block size (threads per block).
- `gridDim.x`: grid size (blocks per grid).
- Question on the slide: what is the largest possible block size? (Reference guide p. 12.)

Explanation

- Strictly speaking, these are **built-in variables**, not functions. Each is a `uint3`/`dim3` with `.x`, `.y` and `.z`. There is also `warpSize` (32).
- They are only defined in device code.
- **Answer: 1024 threads per block** for every compute capability still supported, with per-dimension limits x, y ≤ 1024 and z ≤ 64.
- Exceeding the limit makes the launch fail silently, unless you check `cudaGetLastError()`, which returns "invalid configuration argument".
- **Total threads in the grid:** `gridDim.x * blockDim.x`, which is the stride in the grid-stride loop.

Study instructions

1. Check what "reference guide p. 12" refers to (probably the document on Toledo), and whether the page number still matches the current version.
2. Confirm the 1024 limit with `p.maxThreadsPerBlock` from the query program in the slide-3 notes, and with a `<<<1, 1025>>>` launch followed by `cudaGetLastError()`.

Slide 20 — First session's exercise

What the slide shows

- **Problem:** flip an array (0…8 becomes 8…0) using multiple blocks with multiple threads.
- **Goal:** write and run parallel code on the GPU, migrate memory and verify the result.
- Reports are due by the next lab session.

Explanation

Out-of-place (two buffers) is simple and has no race:

```
__global__ void flip(const int* in, int* out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[n - 1 - i] = in[i];
}
```

In-place is where students fall into a trap. With n threads doing `a[n-1-i] = a[i]`, a thread can overwrite an element before another thread has read it. That is a race, and the result depends on scheduling. The correct version uses only the first half of the threads, each swapping its own pair:

```
__global__ void flip_inplace(int* a, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n / 2) {
        int t = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = t;
    }
}
```

For odd n the middle element stays in place, which is correct. Launch enough blocks for n/2 threads.

Verification on the CPU: check `out[i] == in[n-1-i]` for every i.

Test sizes:

- n = 9 (the slide example, and odd);
- n that is not a multiple of the block size;
- n > 1024, so multiple blocks are required.

Likely mistakes to watch for

- Indexing with only `threadIdx.x` (slide 16).
- No bounds check.
- No `cudaDeviceSynchronize()` before checking with Unified Memory.
- The in-place race described above.
- Verifying only n = 9, where a single block of 9 hides the indexing bug.

Study instructions

1. Solve both versions yourself before the lab.
2. Deliberately run the racy in-place version (`a[n-1-i] = a[i]` with n threads) on a large n and compare it with the correct output. It produces a palindrome-like corruption rather than a flip.
3. Write the one-page report yourself (see slide 9).

Corrections for the Overleaf source

- Title slide: e-mail domain `kueluven.be` should be `kuleuven.be`.
- Title slide: “Academic year 2023/24” needs updating.
- Slide 7: `(nvcc))` has an extra closing parenthesis.
- Slides 9 and 10: “seesions” should be “sessions”.
- Slide 6: the figure predates Ada, Hopper and Blackwell; Pascal and Volta are unsupported in CUDA 13 (see the slide 6 notes).
- Slide 18: “built-in functions” are built-in variables.